

Dashboard de Execução

Rodado agora — 2026-09-08, 18:06–18:13 (horário local). Suíte de testes, stack Docker completo e a API exercitada com cenários reais, incluindo rate limit e propagação de correlation ID.

Tudo conforme o esperado: 36/36 testes, 5/5 serviços saudáveis, 7/7 cenários de API com o resultado previsto.

36/36

testes passando

85,9%

cobertura total

5/5

serviços saudáveis

7/7

cenários de API OK

Testes

python -m pytest, nas quatro variações do manual — sem Docker, SQLite em memória.

COMANDO	RESULTADO	TEMPO
pytest	● 36 passed	4,14s
pytest -m unit	● 23 passed● 13 deselected	1,87s
pytest -m integration	● 13 passed● 23 deselected	1,86s
pytest --no-cov	● 36 passed	1,82s

Cobertura por camada

Perfil típico de arquitetura hexagonal bem testada: domínio e aplicação em 100% (são só regra de negócio e orquestração, fáceis de testar isolados); infraestrutura em 90%; o menos coberto é o "fiação" — `main.py` + `composition.py` — que é composition root, exercitado por e2e mas sem teste unitário dedicado.

app/domain/		100%
app/application/		100%
app/infrastructure/		90%
app/ (main + composition)		55%
TOTAL (320 linhas)		86%

Stack Docker

`docker compose up --build` executado contra o `docker-compose.yml` já corrigido (kafka + alembic). Todos os 5 serviços de pé.

SERVIÇO	IMAGEM	STATUS	PORTA
app	case-test-app	● healthy	8000
consumer	case-test-consumer	● up	—
kafka	bitnamilegacy/kafka:3.7	● healthy	9092
mysql	mysql:8	● healthy	3306
redis	redis:7-alpine	● healthy	6379

Cenários de API

Exercitados via `Invoke-RestMethod/Invoke-WebRequest -UseBasicParsing`, direto contra a stack acima — sem mock.

200	GET /health/ready — MySQL e Redis acessíveis a partir da API. status: ready
201	POST /requests · customer_id=cliente-A, value=1500 → consumer processa → MANUAL_REVIEW id: dde8ad34-cbdc-4225-b257-7e9f1a20b12b
201	POST /requests · customer_id=cliente-B, value=250 → consumer processa → APPROVED id: 3ae4d9cc-6d35-4098-aba9-43c2f0d7d7f6
422	POST /requests · value=-10 → rejeitado antes de tocar no banco "Value error, value deve ser maior que zero"
404	GET /requests/{id} · UUID inexistente "Solicitação não encontrada"

Rate limit em ação

13 `POST /requests` disparados em sequência rápida do mesmo IP (limite configurado: 10/60s). Bloqueou a partir da 6ª desta rajada — não da 11ª — porque a janela é deslizante por IP e já carregava ~5 chamadas dos cenários acima, na mesma janela de 60s. Confirma o comportamento documentado, não é inconsistência.

201 permitido

429 bloqueado

1

2

3

4

5

6

7

8

9

10

11

12

13

Correlation ID ponta a ponta

O ID enviado no header `X-Correlation-Id` da requisição HTTP aparece, idêntico, no log JSON do consumer — dois processos separados, mesma jornada rastreável por um `grep`.

ENVIADO NO POST (CLIENTE-A) X-Correlation-Id: 4a64b560-5b0c-4a14-add4-5b4739da346c	LOG DO CONSUMER, LINHA REAL request_id=dde8ad34... processado com sucesso correlation_id: 4a64b560-5b0c-4a14-add4-5b4739da346c
ENVIADO NO POST (CLIENTE-B) X-Correlation-Id: 04dfcaa6-439f-45cc-bfe0-887bbe2220ae	LOG DO CONSUMER, LINHA REAL request_id=3ae4d9cc... processado com sucesso correlation_id: 04dfcaa6-439f-45cc-bfe0-887bbe2220ae

Stack deixado no ar de propósito (`app`, `consumer`, `mysql`, `redis`, `kafka` saudáveis) pra você continuar explorando pelo Swagger em `http://localhost:8000/docs`. Quando terminar, `docker compose down` encerra tudo.